CS612 · ADVANCED ALGORITHMS · MSC · FALL 2026

Lecture 06
Dynamic Programming I — State-Space Design

The hard part of dynamic programming is not the table. It is deciding what a state must remember.

Saturday 31 October 2026 · 3 hours

PS2 due Sat 07 Nov (Q3 uses today's material)

Motivating problem: comparing sequences

- `git diff` shows the lines that changed between two versions of a file: essentially a **longest common subsequence** of lines.
- Bioinformatics aligns DNA and protein sequences with **edit distance** variants (substitution matrices, gap penalties).
- Spell checkers suggest the dictionary words with the smallest edit distance.

For two sequences of length 10^5, a DP over "all pairs of prefixes" has 10^{10} states: 40 GB as 32-bit integers.

> **DISCUSS · 3 MIN**
>
> If you only need the **distance** (not the alignment), how much of that table do you really need to keep? What if you need the alignment too?

BLOCK A · 00:10–01:00

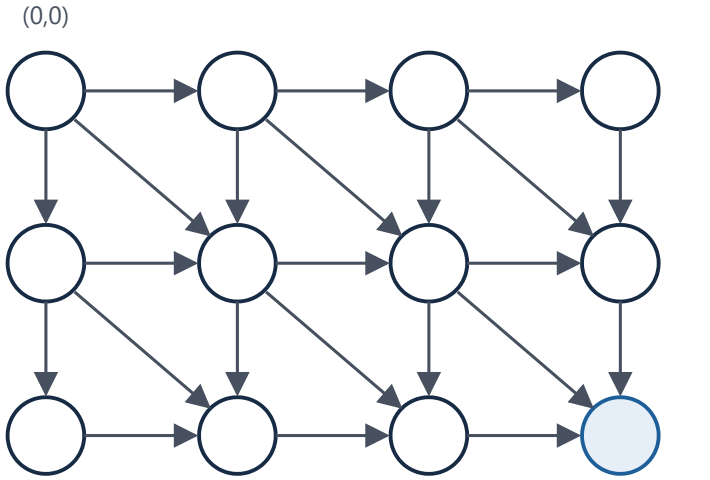
Designing the state

The DAG view · what the state must remember · a correctness template

DP is a path problem on a DAG of states

- **Nodes** = states (subproblems).
- **Edges** = transitions (a decision, with a cost/value).
- **Sources/sinks** = base cases / the goal.
- Optimal value = the best path; counting = the number of paths.
- **Evaluation order** = any topological order of the DAG.
- **Cost** = #states × transition cost = $O(V + E)$ of the DAG.

> A cyclic dependency means it is not DP (as stated): either the state is missing information, or the problem is a genuine shortest path with cycles (L10).



prefix-pair states of LCS / edit distance

What must the state remember?

Principle: the state must contain exactly the information about the past that the **remaining** decisions and their costs depend on. The rest of the history must be irrelevant (a "Markov" property).

Example. Climb n stairs with steps of 1 or 2, **never two 2-steps in a row**. Count the ways.

- State "i stairs done"? Not enough: whether a 2-step is allowed next depends on the last step.
- State (i, ℓ), with $\ell \in \{1, 2\}$ the last step: $W(i, 1) = W(i-1, 1) + W(i-1, 2)$, $W(i, 2) = W(i-2, 1)$.

TEST FOR A CANDIDATE STATE

Take two different histories that lead to the same state. Must every continuation have the same value (cost, feasibility) for both? If not, the state is missing information.

AND THE OPPOSITE ERROR

Remembering too much (e.g. the full set of items chosen) makes the state space exponential. Good design removes everything the future does not need.

A correctness template for DP

1. **State semantics:** in words, what $D[\text{state}]$ equals (e.g. "the length of an LCS of $X[1..i]$ and $Y[1..j]$").
2. **Recurrence** by case analysis on the **last decision** of an optimal solution.
3. **Correctness:**
   a. *Soundness:* every value the recurrence produces is achievable (each case builds a valid solution);
   b. *Optimality:* an optimal solution falls into one of the cases, and its remainder is optimal for the smaller state (**cut-and-paste**).
4. **Base cases** that match the semantics.
5. **Order:** a topological order of the dependencies (or memoization).
6. **Answer extraction** from the final state(s).
7. **Complexity:** #states × cost per transition; space and how to reduce it.
8. **Reconstruction** of an optimal solution (store choices, or re-derive them).

Exercise A: define the state (no code)

1. **Word break.** Given a string $s[1..n]$ and a dictionary, can s be split into dictionary words? Give the state, its meaning, the recurrence and the cost (with dictionary lookup cost ℓ).
2. **Grid with few turns.** Count monotone (right/down) paths from $(1,1)$ to (n,n) that change direction **at most k times**. What must the state remember? How many states?
3. **Deadline tasks.** Tasks $1..n$ must be done in order. Task i takes t_i and can be skipped at penalty p_i. Total time must not exceed T. Minimise the penalty. What is the state, and is the algorithm polynomial?

Pairs · 10 min · then apply the two-histories test to each of your states.

BLOCK B · 01:10–02:00

Three deep case studies

LCS · edit distance · matrix-chain multiplication

LCS: state, recurrence, and the key lemma

State: $L[i][j]$ = the length of a longest common subsequence of $X[1..i]$ and $Y[1..j]$.

$$L[i][j] = \begin{cases} 0 & i = 0 \text{ or } j = 0 \\ L[i-1][j-1] + 1 & x_i = y_j \\ \max\big(L[i-1][j],\, L[i][j-1]\big) & x_i \neq y_j \end{cases}$$

LEMMA (THE MATCH CASE IS SAFE)

If $x_i = y_j$, some LCS of $X[1..i], Y[1..j]$ uses x_i matched with y_j.

PROOF (EXCHANGE)

Take any LCS Z and let its last matched pair be $(x_{i'}, y_{j'})$, with $i' \le i$, $j' \le j$. If $i' < i$ and $j' < j$, appending the pair (x_i, y_j) gives a longer common subsequence: a contradiction. Otherwise replace the last pair by (x_i, y_j): still increasing, same length. So some LCS ends with x_i matched to y_j, and by cut-and-paste its remainder is an LCS of $X[1..i-1], Y[1..j-1]$.

■

LCS: the table, reconstruction, and space

	∅	B	D	C	A	B
∅	0	0	0	0	0	0
A	0	0	0	0	1	1
B	0	1	1	1	1	2
C	0	1	1	2	2	2
B	0	1	1	2	2	3

$X =$ ABCB, $Y =$ BDCAB: LCS length 3 (e.g. **BCB**).

Reconstruction: from (n, m), follow the choice that produced each value: diagonal on a match, otherwise to the larger neighbour. $O(n + m)$ steps, but it needs the whole table.

Goal	Time	Space
Length only (two rows)	$\Theta(nm)$	$\Theta(\min(n, m))$
Length + an LCS (full table)	$\Theta(nm)$	$\Theta(nm)$
Length + an LCS (Hirschberg)	$\Theta(nm)$ **(≈ 2×)**	$\Theta(n + m)$

Edit distance

State: $E[i][j]$ = the minimum cost of transforming $X[1..i]$ into $Y[1..j]$, with insertion, deletion and substitution costing 1 each (a match costs 0).

$$E[i][j] = \min \begin{cases} E[i-1][j] + 1 & \text{delete } x_i \\ E[i][j-1] + 1 & \text{insert } y_j \\ E[i-1][j-1] + [x_i \neq y_j] & \text{match/substitute} \end{cases}$$

Base cases: $E[i][0] = i$, $E[0][j] = j$.

Correctness: write an optimal edit sequence as an **alignment**. Its last column is $(x_i, -)$, $(-, y_j)$ or (x_i, y_j). These are exactly the three cases, and the rest of the alignment is optimal for the smaller state (cut-and-paste).

DAG VIEW

A shortest path from $(0, 0)$ to (n, m) in a grid graph with edge weights 0/1. That makes it a special shortest-path problem, and this structure is what Myers' $O((n+m)D)$ algorithm exploits.

General costs: any non-negative costs $c_{ins}(y)$, $c_{del}(x)$, $c_{sub}(x, y)$ work unchanged (sequence alignment in bioinformatics). Affine gap penalties need extra state: "am I inside a gap?" (Gotoh).

Matrix-chain multiplication: interval states

$A_1 A_2 \cdots A_n$, with A_i of size $p_{i-1} \times p_i$. Multiplying $p \times q$ by $q \times r$ costs pqr. Choose the parenthesisation that minimises the total cost.

State: $M[i][j]$ = the minimum cost to compute $A_i \cdots A_j$.

$$M[i][j] = \min_{i \le k < j} \Big(M[i][k] + M[k+1][j] + p_{i-1} p_k p_j \Big), \quad M[i][i] = 0.$$

Last decision: the *final* multiplication splits the chain at some k.

Order: by increasing chain length $j - i$.

States	Transition	Time	Space
$\Theta(n^2)$	$O(n)$	$\Theta(n^3)$	$\Theta(n^2)$

ASK

Dimensions $10 \times 100,\ 100 \times 5,\ 5 \times 50$. Compare $(A_1 A_2) A_3$ with $A_1 (A_2 A_3)$.

Greedy "multiply the cheapest adjacent pair first" is **not** optimal (exercise). The number of parenthesisations is a Catalan number: $\Omega(4^n / n^{3/2})$.

How good are these algorithms? A hardness remark

Problem	Classic DP	Known better?
LCS, edit distance (general strings)	$\Theta(nm)$	$O(n^2/\log^2 n)$ (Four Russians; Masek–Paterson). No $O(n^{2-\varepsilon})$ algorithm unless the Strong Exponential Time Hypothesis fails (Backurs–Indyk 2015; Abboud et al. 2015).
Edit distance, small distance D	$\Theta(nm)$	$O((n+m)D)$ (Ukkonen; Myers), $O(n+D^2)$ (Landau–Vishkin)
Matrix-chain order	$\Theta(n^3)$	$O(n \log n)$ (Hu–Shing)

MSC POINT

"Conditional lower bounds" (fine-grained complexity) explain why decades of effort have not produced strongly subquadratic edit-distance algorithms. It is the polynomial-time analogue of NP-hardness (L13).

Exercise B: prove and refute

1. Find dimensions $p_0..p_3$ (3 matrices) for which "multiply the cheapest adjacent pair first" is not optimal.
2. Prove the cut-and-paste step for MCM: if the optimal final split is at k, then the parenthesisation used for $A_i..A_k$ is optimal for that subchain.
3. Edit distance: what is E for "kitten" → "sitting"? Give one optimal alignment.

Individually · 7 min

BLOCK C · 02:10–02:55

State-design workshop

Design, then critique

Exercise C1: design (groups)

1. **Longest common substring** (contiguous) of X and Y. Why does the LCS state not work unchanged? Give the right state and complexity.
2. **Cutting a stick.** A stick of length ℓ must be cut at given positions $c_1 < \cdots < c_k$. Each cut costs the length of the piece being cut. Minimise the total cost. (Hint: which cut happens *last*… or first?)
3. **Segmented least squares (sketch).** Fit points $1..n$ with line segments; cost = the sum of the segments' squared errors + C per segment. State and recurrence. What is the cost if each segment's error is computed from scratch?

Groups · 15 min · for each: state in words, recurrence, base, order, complexity

Exercise C2: critique these DP proposals

1. **Circular robber.** Houses $1..n$ are on a *circle* (n is adjacent to 1). Maximise the loot, never robbing two adjacent houses. Proposal: $R[i] = \max(R[i-1], R[i-2] + v_i)$, answer $R[n]$. What is wrong? Fix it without adding a dimension.
2. **Longest increasing subsequence.** Proposal: $L[i]$ = the length of an LIS of $a[1..i]$; $L[i] = L[i-1] + 1$ if $a_i > a_{i-1}$, else $L[i-1]$. Give a counterexample and a correct state.
3. **Grid path with a budget.** Maximise the collected value on a monotone path, with the total path *cost* $\leq B$. Proposal: $V[i][j]$ = the best value at (i, j), keeping only paths within budget. What does the two-histories test say?

Groups · 10 min

Synthesis

1. DP = paths in a DAG of states: order = topological order; cost = #states × transition cost.
2. The state must remember exactly what the future depends on. Apply the **two-histories test**.
3. Correctness = state semantics + case analysis on the last decision + cut-and-paste + base cases.
4. LCS/edit distance: prefix-pair states, $\Theta(nm)$; linear space for values, Hirschberg for alignments. MCM: interval states, $\Theta(n^3)$.
5. "There is a DP" does not mean "this is the best algorithm": fine-grained lower bounds, and better special-case algorithms.

NEXT WEEK · L07 (SAT 07 NOV)

Harder state modelling (LIS in $n \log n$, weighted interval scheduling), state and transition optimisation, space optimisation, dominance, 0/1 knapsack and **pseudo-polynomial time**: the bridge to NP-hardness.

Read: KT 6.1, 6.4; CLRS 4e 14 problems. **PS2 due Sat 07 Nov.** Midterm (L01–L07) working date Sat 14 Nov, separate exam slot.